

J.A.R.V.I.S. — Phase 4 (v4)

Implementation Plan | Quality-Driven | Cognitive Architecture | Modular Monolith

Phase 1 Planning	Phase 2 Requirements	Phase 3 Design	Phase 4 Implementation	Phase 5 Testing	Phase 6 Deployment
------------------	----------------------	----------------	------------------------	-----------------	--------------------

v4 changes from v3: (1) core/memory.py now uses Mem0 as the unified API layer wrapping ChromaDB and knowledge graph. (2) core/knowledge_graph.py added as a new module with NetworkX, spreading activation, and automatic edge writing. (3) core/episodic_store.py added for timestamped event logging. (4) core/consolidation.py added for nightly memory consolidation with forgetting curve. (5) core/context_budget.py added for token budget enforcement using tiktoken. (6) All 8 memory system connections are traced through the steps.

1. Quality Attribute Priority Ranking

R a n k	Quality Attribute	Definition	Sub-characteristics	Impact on Jarvis
1	Reliability	Consistent function without failure	Maturity, Availability, Fault Tolerance, Recoverability	Jarvis runs 24/7. Voice fallback always works. Every API and memory failure caught.
2	Maintainability	Easily modified to fix bugs or adapt	Modularity, Reusability, Analysability, Testability	Modular monolith: one module change never breaks another. Test stub per step.
3	Reusability	Asset usable in more than one system	Sub-characteristic of Maintainability	tools/ modules and memory modules importable as standalone libraries.
4	Usability	Ease of learning, operation, interaction	Learnability, Operability, User error protection	Single command startup. Natural voice. Confirmation gates. Context-aware responses.
5	Performance Efficiency	Resource use vs response time	Time behaviour, Resource utilisation	Wake word <500ms. E2E <5s. Memory retrieval <300ms. Context budget enforced.
6	Security	Protection of information and data	Confidentiality, Integrity, Authenticity	Keys in .env. No audio stored. Path sandboxing. Email + destructive action gates.

7	Portability	Transfer to another environment	Adaptability, Installability	Pinned requirements.txt. .env for secrets. README for setup. Linux port feasible.
8	Scalability	Handles increased workload	Horizontal/vertical scale	Modular monolith: new tools added without touching core. Caches prevent spikes.

2. Implementation Steps

Steps are ordered to build the most stable foundation first. Every step must pass all its acceptance criteria before the next begins. Every step includes: modular monolith boundary, architecture context, code reference, guidelines, and measurable acceptance criteria.

STEP 01	Project setup & environment	/, requirements.txt, .env, config.json, README.md
Priority: Critical	Modular monolith boundary: Creates the single deployable unit. All modules in one jarvis/ package. One process. No microservices.	

Create full folder structure, virtual environment, requirements.txt with ALL pinned dependencies (including mem0ai, networkx, tiktoken, APScheduler, webRTCvad, psutil, BeautifulSoup4, PyPDF2, colorama), .env with all 7 required keys, config.json with all configuration including competence maps template, README.md with 5 sections, .gitignore.

Implementation guidelines:

- requirements.txt must include: mem0ai, networkx, tiktoken, APScheduler, webRTCvad, psutil, BeautifulSoup4, PyPDF2, youtube-transcript-api — all pinned
- config.json must include: competence_map template per agent, allowed_apps, blocked_paths, daily_email_limit, memory_max_turns, supported_symbols, activation_decay_rate (0.05), activation_prune_threshold (0.1)
- .env must have: ANTHROPIC_API_KEY, PICOVOICE_API_KEY, ELEVENLABS_API_KEY, ELEVENLABS_VOICE_ID, ALPHA_VANTAGE_KEY, GMAIL_ADDRESS, GMAIL_APP_PASSWORD
- README.md: (1) Prerequisites, (2) venv setup, (3) API key sources, (4) Gmail App Password walkthrough, (5) How to run
- Write tests/test_setup.py: verify folders, .env loads, config.json parses, mem0ai imports successfully

Acceptance criteria:

- ✓ pip install -r requirements.txt completes on clean Python 3.11 venv
- ✓ import mem0 and import networkx succeed without errors
- ✓ All folder paths from Phase 3 design exist
- ✓ pytest tests/test_setup.py passes

STEP 02	core/llm.py — LLM wrapper	core/llm.py
Priority: Critical	Modular monolith boundary: ONLY file that imports anthropic. Exposes ask(messages, tools) and ask_simple(prompt). All other modules call these two functions.	

Claude API wrapper with full tool_use schema, exponential backoff retry (1s/2s/4s), per-call token logging, and a system prompt that instructs the model to use [REMEMBER: fact] tags for memory-worthy information.

Implementation guidelines:

- TOOL_DEFINITIONS: 5 schemas (pc_control, web_search, trading_data, send_email, coding_assist) as module-level constant
- System prompt instructs: respond concisely for voice (<3 sentences), use tool_use for actions, tag memories with [REMEMBER: fact]
- Retry: catch RateLimitError (wait 60s + 1 retry), APIStatusError (backoff 3x), APIConnectionError (backoff 3x), AuthenticationError (fail fast)

- Log every call: timestamp, model, input_tokens, output_tokens, latency_ms, tool_used → data/logs/llm_calls.log
- Write tests/test_llm.py: mock anthropic client, assert tool_use returned for action commands, assert retry fires on RateLimitError

Acceptance criteria:

- ✓ ask() returns tool_use block with name='pc_control' for 'open chrome' prompt
- ✓ RateLimitError triggers exactly 3 retries then returns None (verified by mock count)
- ✓ Every call appends a line to data/logs/llm_calls.log
- ✓ pytest tests/test_llm.py passes

STEP 03	core/memory.py — Unified cognitive memory	core/memory.py +
		core/knowledge_graph.py + core/episodic_store.py
Priority: Critical	Modular monolith boundary: The entire cognitive architecture lives here. ONLY core/orchestrator.py calls these modules. No tool module imports memory. This is the hippocampus of the system.	

Implement the full cognitive memory architecture: (1) Mem0 unified API for semantic + associative memory, (2) NetworkX knowledge graph with spreading activation, (3) SQLite episodic store, (4) context budget assembly using tiktoken, (5) [REMEMBER:] tag extraction from LLM responses. This is the most important step in the entire system.

Implementation guidelines:

- Mem0: mem0 = Memory() — uses ChromaDB as backend automatically. Call memory.add(text, user_id=agent_id) and memory.search(query, user_id=agent_id)
- Knowledge graph: networkx.DiGraph(). add_edge(subject, predicate, object) for every agent action. get_neighbours(node, hops=2) for spreading activation
- Episodic store: SQLite table (id, timestamp, agent_id, task_id, action, inputs_json, outputs_json, success_score). Insert on every agent action completion
- Context budget: tiktoken.encoding_for_model('cl100k_base'). Count tokens per section. If section exceeds budget, call ask_simple('Summarise in 200 words: ...')
- Context assembly order: system_prompt(2k) + memory_recall(1.5k) + graph_neighbours(1.5k) + episodic_summary(1k) + project_record(1.5k) + task_context(5k) + conversation(10k) + user_input(1k)
- [REMEMBER:] extraction: regex r'\[REMEMBER:\s*(.+?)\]' on every LLM response. Call memory.add() for each match. Also write Agent -LEARNED-> Fact edge to knowledge graph
- Write tests/test_memory.py: assert mem0.add() stores fact, assert search() returns it, assert knowledge graph has correct edges, assert context budget never exceeds 180k tokens

Acceptance criteria:

- ✓ memory.add('user prefers dark mode', user_id='jarvis') stores retrievable fact
- ✓ memory.search('colour preferences', user_id='jarvis') returns the stored fact
- ✓ Knowledge graph: add_edge('Senior_Dev', 'BUILT', 'auth.py') — query returns edge
- ✓ Spreading activation: query 'auth.py' returns auth.py + all 2-hop neighbours
- ✓ Context assembly never exceeds 180,000 tokens (tiktoken verified)
- ✓ Episodic store inserts row correctly and queries by agent_id and date range
- ✓ pytest tests/test_memory.py passes all scenarios

STEP 04	input/wake_word.py — Wake word detection	input/wake_word.py
Priority: Critical	Modular monolith boundary: Daemon thread — communicates only via threading.Event callback. Never imports from tools/ or output/.	

Porcupine wake word listener in a daemon thread. threading.Event stop signal. Auto-recovery from PyAudio stream errors. Zero audio written to disk. Latency measured and logged per detection.

Implementation guidelines:

- WakeWordListener class: __init__(key, callback), start(), stop(). Daemon thread, threading.Event stop signal
- stream.read() wrapped in try/except Exception — log error type, sleep(0.1), continue loop (auto-recovery)
- Measure latency: time.perf_counter() before and after callback fires. Log [PERF] wake_latency=Xms
- Zero audio bytes written to disk — pcm is a RAM tuple only
- Write tests/test_wake_word.py: mock porcupine.process() returns 0 on 3rd call, assert callback fires once, assert stop() joins within 2s

Acceptance criteria:

- ✓ Wake word detected within 500ms (logged by perf_counter)
- ✓ No audio files created anywhere on disk
- ✓ Thread joins cleanly within 2 seconds of stop()
- ✓ PyAudio exception does not crash thread — loop continues after 100ms
- ✓ pytest tests/test_wake_word.py passes

STEP 05	input/speech_to_text.py — Transcription	input/speech_to_text.py
Priority: Critical	Modular monolith boundary: Triggered by wake word Event. Runs synchronously in main loop. Returns plain string to orchestrator — no audio data leaves this module.	

webrtcvad (aggressiveness=2) stops recording exactly when speech ends — 10 consecutive silent 30ms frames = stop. Whisper 'base' model loaded once at startup (not per call). fp16=False for CPU-only Windows machines.

Implementation guidelines:

- webrtcvad aggressiveness=2. FRAME_MS=30. SILENCE_THRESHOLD=10 frames (300ms). MAX_FRAMES=300 (9s)
- Whisper 'base' model loaded at jarvis.py startup via load_model(). Never reloaded per transcription
- fp16=False always — CUDA not assumed. Convert int16 PCM to float32 numpy before passing to whisper.transcribe()
- Empty audio (no speech) returns "" — orchestrator checks and skips LLM call
- Write tests/test_stt.py: mock stream bytes, mock whisper model, assert correct lowercase string returned

Acceptance criteria:

- ✓ 3-second sentence transcribed within 3 seconds on CPU
- ✓ Silence returns "" without calling Whisper
- ✓ fp16=False set — no CUDA errors on CPU machines
- ✓ pytest tests/test_stt.py passes

STEP 06	output/voice.py + output/display.py	output/voice.py output/display.py
Priority: High	Modular monolith boundary: ONLY modules producing audio and visual output. All others call speak(text) and log(level, msg).	

voice.py: ElevenLabs online + pytsx3 offline fallback. threading.Lock prevents overlap. LRU cache (10 entries) for common phrases. Voice ID from config.json. display.py: colorama ANSI colours (INFO=cyan, SUCCESS=green, WARNING=yellow, ERROR=red). Non-blocking toast notifications. Session log file per run.

Implementation guidelines:

- voice.py: ELEVENLABS_VOICE_ID from config.json (default: 21m00Tcm4TlvDq8ikWAM Rachel). Document 5 free voice IDs in README
- Internet check: socket connect to 8.8.8.8:53 with 1s timeout — faster than HTTP
- ElevenLabs failure: catch all exceptions, silently fall back to pytsx3, log WARNING
- display.py: colorama.init(autoreset=True). Thread-safe file writes (threading.Lock). Log format: [HH:MM:SS] [LEVEL] message
- Write tests/test_voice.py: mock _is_online()=False → assert pytsx3 called. Mock ElevenLabs raise → assert fallback. Write tests/test_display.py: assert log file written

Acceptance criteria:

- ✓ speak('hello') plays ElevenLabs audio when online
- ✓ ElevenLabs failure silently switches to pytsx3 — no exception, no silence
- ✓ Concurrent speak() calls serialised by Lock — no overlapping audio
- ✓ log('INFO', 'test') writes correct format to session log file
- ✓ pytest tests/test_voice.py and tests/test_display.py pass

STEP 07	core/orchestrator.py — Command routing	core/orchestrator.py
Priority: Critical	Modular monolith boundary: Spine of the monolith. ONLY module importing from both core/ and tools/. TOOLS registry is the single extension point. Adding a tool = 1 file + 2 lines.	

Receives transcribed text → assembles context (memory + graph + episodic) → calls LLM → dispatches tool → collects result → generates voice reply → writes episodic event → updates knowledge graph edges. Confirmation gate for destructive actions (send_email, delete_file).

Implementation guidelines:

- TOOLS = {'pc_control': pc_control.execute, 'web_search': web_search.execute, ...} — module-level dict
- Before LLM call: memory.build_context(user_text) assembles the full 8-section prompt
- After LLM response: episodic_store.insert(agent='jarvis', action='tool_use', tool=name, result=result, success=True)
- After tool execution: knowledge_graph.add_edge('Jarvis', 'EXECUTED', tool_name) with timestamp metadata
- Confirmation gate: DESTRUCTIVE = {'send_email', 'delete_file'}. Call speak() + record_and_transcribe(). 2 attempts. Second unclear = cancel
- Tool exception: catch all, log ERROR, return user-friendly string. Never propagate
- Write tests/test_orchestrator.py: mock llm.ask() returns tool_use for 'open chrome', assert pc_control.execute() called with correct inputs

Acceptance criteria:

- ✓ handle_command('open chrome') dispatches to pc_control.execute(action='open_app', target='chrome')
- ✓ handle_command('what is a for loop') returns text without calling any tool

- ✓ Tool exception returns friendly error string — no crash
- ✓ send_email triggers confirmation gate — 'no' cancels without sending
- ✓ Episodic store receives an entry after every command
- ✓ pytest tests/test_orchestrator.py passes

STEP 08	tools/ — All 5 tool modules	tools/pc_control.py · web_search.py
		· trading_tool.py · coding_tool.py · email_tool.py
Priority: High	Modular monolith boundary: Each tool is self-contained. execute(inputs: dict) -> str. No tool imports another tool. Each independently testable and reusable.	

pc_control: psutil for close_app, os.startfile for open_app, ALLOWED_APPS whitelist, _is_safe_path() sandboxing, FAILSAFE=True. web_search: DuckDuckGoSearchException handling, 10s rate limit wait + 1 retry, LRU cache(20). trading_tool: yfinance fast_info, Alpha Vantage fallback + news, .LG NGX tickers, FR-12 DEFERRED (TODO comment only). coding_tool: action enum (qa/review), 50KB limit, ALLOWED_EXTENSIONS whitelist. email_tool: Gmail SMTP App Password, _await_confirmation() 2 attempts, daily limit.

Implementation guidelines:

- pc_control: ALLOWED_APPS from config.json. _is_safe_path() uses os.path.realpath() before checking against home dir. pyautogui.FAILSAFE=True at module level
- web_search: import DuckDuckGoSearchException from duckduckgo_search.exceptions. On exception: sleep(10), retry once, then return graceful string
- trading_tool: FR-12 TODO comment at top of file. yfinance Ticker.fast_info (not Ticker.info). Cache: prices 60s, news 10min
- coding_tool: LLM tool schema action enum ['qa','review']. For review: read file, pass to ask_simple() with coding system prompt. Full text to terminal, summary to voice
- email_tool: GMAIL_ADDRESS and GMAIL_APP_PASSWORD from os.getenv() ONLY. Confirmation keywords: yes=['yes','send','confirm','do it']. No=['no','cancel','stop','abort']
- Write test stubs for all 5: test_pc_control.py, test_web_search.py, test_trading.py, test_coding.py, test_email.py

Acceptance criteria:

- ✓ pc_control: open whitelist app succeeds, open unknown app returns denial, System32 access blocked
- ✓ web_search: DuckDuckGoSearchException triggers 10s wait + retry, second failure returns graceful message
- ✓ trading_tool: returns price for AAPL, BTC-USD, EURUSD=X, DANGCEM.LG. FR-12 TODO present, no RSI code
- ✓ coding_tool: 'open chrome.exe' returns extension denial, file >50KB returns size warning
- ✓ email_tool: 'no' cancels send, SMTP failure returns friendly error string
- ✓ All 5 test stubs pass with pytest

STEP 09	core/consolidation.py — Memory consolidation	core/consolidation.py
Priority: High	Modular monolith boundary: Background async task. Runs on APScheduler at 2am daily. Reads episodic_store, writes to memory (Mem0) and knowledge_graph. Never called from the main voice loop.	

Nightly consolidation job implementing the hippocampal consolidation pattern: (1) read today's episodic events, (2) LLM extracts durable facts, (3) store facts in Mem0 semantic memory, (4) update knowledge graph edges, (5) apply forgetting curve decay (×0.95 daily) to all memory activation scores, (6) archive memories below 0.1 threshold to data/logs/archived_memories.jsonl.

Implementation guidelines:

- APScheduler: BackgroundScheduler(). scheduler.add_job(consolidate, 'cron', hour=2). scheduler.start() in jarvis.py startup
- Forgetting curve: query all ChromaDB entries, multiply activation_score metadata by 0.95, update. Use ChromaDB update() method
- Consolidation LLM prompt: 'From these agent events today, extract 5-10 facts worth remembering long-term as short declarative sentences. Return as JSON list.'
- Archive: entries with activation_score < 0.1 are moved to archived_memories.jsonl (not deleted — recoverable)
- Log consolidation run: [CONSOLIDATION] events_processed=X facts_extracted=Y memories_archived=Z duration=Xs
- Write tests/test_consolidation.py: mock episodic_store, assert facts extracted and stored, assert activation scores decayed

Acceptance criteria:

- ✓ Nightly job runs at 2am without blocking the voice loop
- ✓ Episodic events from today produce at least 1 extracted fact in Mem0
- ✓ All activation scores reduced by 5% after each run
- ✓ Memories below 0.1 archived to archived_memories.jsonl
- ✓ Consolidation log entry written with correct metrics
- ✓ pytest tests/test_consolidation.py passes

STEP 10 jarvis.py — Main entry point jarvis.py	
Priority: Critical	Modular monolith boundary: Composition root — the ONLY file that imports from all layers. Wires everything together. Starts the single-process monolith.

Startup: validate .env (7 keys), load config, run health check, load Whisper, start WakeWordListener thread, start APScheduler, print banner, speak ready message. Main loop: wait for wake_word Event → transcribe → build_context → orchestrate → speak → log → measure e2e latency. Shutdown: Ctrl+C → stop threads → flush logs → run final consolidation → speak goodbye.

Implementation guidelines:

- validate_env(): check all 7 .env keys. For each missing: print key name + URL to get it. sys.exit(1) if any missing
- health_check(): lightweight ping to each API concurrently (threading). Log SUCCESS/FAILED per service. Non-blocking — Jarvis starts regardless
- E2E timing: t0 = time.perf_counter() after wake_word fires. Stop after speak() returns. Log [PERF] e2e=Xms. Warning if >5000ms (NFR-02 breach)
- APScheduler: start consolidation scheduler before main loop. Runs nightly at 2am in background
- --debug flag: argparse. Sets log level to DEBUG in display.py. Enables LLM token count logging
- All 13 test stub files must exist and pass pytest before Phase 5 begins

Acceptance criteria:

- ✓ python jarvis.py reaches 'Ready' state within 10 seconds
- ✓ Ctrl+C: clean shutdown, goodbye spoken, no orphaned threads
- ✓ Missing .env key: correct key name + URL printed, sys.exit(1)
- ✓ [PERF] e2e=Xms logged after every command. Warning fires if >5000ms
- ✓ APScheduler running — consolidation job scheduled for 2am
- ✓ All 13 test stubs pass with pytest